QUENYX VOPS HUB

INTERNAL — OPERATIONS

Operations Runbook

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.1

Document Version	2.1
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Internal — Operations
Owner	Operations / SRE
Status	Released
Last Updated	2026-06-30
Document Type	Operations runbook

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0 RC1; native monitoring operations; Unified AI Workspace operations. See also `docs/OBSERVE_RUNBOOK.md`.
2.1	2026-06-30	Added Operations Intelligence (Sprint 21) operations — shares the Quenyx AI runtime; no new services to operate.

Table of Contents

18 — Operations Runbook

Audience: Ops / SRE. **Scope:** Run, monitor, and recover a Quenyx vOPS HUB deployment. Commands assume the backend dir and a Linux host (adjust paths).

---

1. Daily checks

- [ ] `curl -s https://<host>/api/health` → healthy; gateway `/health` → `{"status":"ok"}`.
- [ ] Scheduler heartbeat: `tail backend/storage/logs/scheduler.log` shows recent `observe:run-checks`.
- [ ] Queue worker alive: `systemctl status quenyx-queue`.
- [ ] Error log scan: `tail -n 200 backend/storage/logs/laravel.log` for ERROR/exception spikes.
- [ ] Disk and DB connectivity OK.

2. Weekly checks

- [ ] Review `audit-logs` for unexpected privileged actions.
- [ ] Verify backups exist and are restorable (spot-restore to staging).
- [ ] Check SSL certificate expiry.
- [ ] Review failed jobs: `php artisan queue:failed`.
- [ ] Confirm AI flags still match intended posture (see §11/§12 below).

3. Backups

- **DB:** `mysqldump -u <user> -p <db> > backup-$(date +%F).sql` (automate nightly, encrypt, offsite).
- **Storage:** archive `backend/storage/` (agent binaries, artifacts).
- **Secrets:** `.env` lives in a secrets manager, not in backups-in-the-clear.

4. Restores

- **DB:** `mysql -u <user> -p <db> < backup-YYYY-MM-DD.sql`.
- **Storage:** extract the archive to `backend/storage/`.
- Re-run `php artisan config:cache` after restore; verify health.

5. Logs

- Laravel: `backend/storage/logs/laravel.log`; scheduler: `scheduler.log`.
- Gateway: service logs via `journalctl -u quenyx-gateway`.
- Nginx: access/error logs under `/var/log/nginx/`.

6. Queues

- Status: `systemctl status quenyx-queue`; restart: `systemctl restart quenyx-queue`.
- Retry failed: `php artisan queue:retry all`; inspect: `php artisan queue:failed`.
- Required for **port scans** and **RAG indexing jobs**.

7. Scheduler

- Cron must run `php artisan schedule:run` every minute (`crontab -u www-data -l`).
- If monitoring shows "never": confirm cron, PHP path, and `storage/logs` writability.

8. SSL

- Renew certs (e.g. certbot); reload Nginx: `systemctl reload nginx`.
- Verify HTTP→HTTPS redirect and HSTS as configured.

9. DB health

- `php artisan migrate:status` → all applied.
- Monitor connections, slow queries, and disk. Alert on connectivity loss.

10. Cache

- Clear after config changes: `php artisan config:clear` (dev) / `config:cache` (prod).
- Route/view caches: `php artisan route:cache`, `view:cache`. Clear all: `php artisan optimize:clear`.

11. Incident handling

1. Confirm scope via health endpoints + logs.
2. Identify the failing component (backend/gateway/db/queue/scheduler).
3. Mitigate (restart service, fail over, disable a feature flag).
4. Capture logs and audit entries.
5. Post-incident: file the fix forward (new migration/code), update this runbook.

12. Rollback

- **App:** deploy previous tag; rebuild backend/frontend/gateway; restart.
- **DB:** restore from pre-deploy dump (don't `migrate:rollback` destructive changes in prod).
- **Corpus:** roll back to the previous active revision (deterministic snapshot).

13. Emergency: disable AI

```
# In backend/.env
AI_ENABLED=false
AI_PROVIDER=mock
RAG_ENABLED=false
AI_COPILOT_RAG_ENABLED=false
php artisan config:cache   # or config:clear in dev
```

This forces mock mode — **no external model calls** — while keeping deterministic features working.

14. Emergency: disable a module

- Per workspace: revoke via `PUT /api/workspaces/{project}/modules/{moduleKey}/override` (audited).
- QynSight gate: remove the `qynsight` entitlement to disable `observe/*` for a workspace.
- Frontend: a hidden module stays hidden via the existing flag (no action needed).

15. Troubleshooting commands

```
php artisan about                # environment summary (needs mbstring)
php artisan route:list           # verify routes/no collisions
php artisan migrate:status       # migration state
php artisan queue:failed         # failed jobs
php artisan optimize:clear       # clear caches
tail -f backend/storage/logs/laravel.log
journalctl -u quenyx-gateway -f
```

Quenyx AI (Unified AI Workspace — Sprint 20) — operations

> **RC1.1:** UI label is **Quenyx AI**. Routes unchanged (`/api/ai/*` , `/ai-workspace/*` ; `/quenyx-ai/*` redirects). The provider list is now catalog-driven and the dev-only **mock** provider is hidden outside `local` / `testing` ; the production default provider is **OpenAI** when configured, otherwise an honest "no provider configured" state (never mock).

Deploy (after pulling):

```
cd backend
composer install --no-dev --optimize-autoloader   # or: composer dump-autoload -o
php artisan migrate --force                         # adds projects.uuid (backfilled),
                                                    # ai_prompt_templates, ai_provider_settings,

php artisan route:list | grep ai                    # expect /api/ai/workspace/summary, conversat
php artisan config:cache
```

- **Enable/disable** the surface: `AI_WORKSPACE_ENABLED` (default `true`). When `false`, all `/api/ai/*` workspace endpoints return 404 and the sidebar item leads to an empty surface.
- **Safe by default**: with `AI_ENABLED=false` (default), chat uses the mock provider; nothing reaches an external model. No raw provider secrets are stored — `ai_provider_settings.settings` is encrypted (depends on a valid `APP_KEY`; rotating `APP_KEY` invalidates stored secrets, which must be re-entered).
- **Default provider (RC1.1)**: leave `AI_PROVIDER` unset to let the registry resolve it safely (OpenAI when `OPENAI_API_KEY` is set; `mock` only in local/testing; otherwise none). Set `AI_PROVIDER` explicitly to override. The provider **catalog** is informational; only providers with a real adapter (today OpenAI) are executable. The **Test connection** action / `POST /api/ai/providers/{uuid}/test` runs a genuine `health()` for executable providers and is audited.
- **Cost tracking** shows currency only when `ai.workspace.pricing` is configured; otherwise it is token-only by design (no fabricated amounts).
- **Auditing**: AI conversations, provider/template/permission changes are written to `audit_logs` (`action LIKE 'ai%'`); surface them via the Activity/Notifications tabs or query the table directly.
- **Rate limiting**: `throttle:ai-workspace` (default 120/min, `AI_WORKSPACE_RATE_LIMIT`).

QynSight Operations Intelligence (Sprint 21) — operations

Operations Intelligence is an **additive layer** that reuses the Quenyx AI runtime — there is **no new daemon, queue, or service to operate**. It ships with the same backend deploy (no new migrations; UUIDs are derived deterministically at runtime). After pulling:

```
cd backend
composer dump-autoload -o
php artisan route:list | grep qynsight/intelligence   # expect overview, copilot, alerts/ … /exp
php artisan config:cache
```

- **Prerequisites**: the workspace must have the `qynsight` entitlement and the `can_use_ai` AI capability; the requesting member needs monitoring RBAC. Live operational data still requires the **scheduler** (`observe:run-checks`, `observe:evaluate-alerts`) and the **queue worker** (port scans) per §6/§7 — Operations Intelligence reads that data and never fabricates it.
- **AI posture**: governed by the same flags as Quenyx AI. With `AI_ENABLED=false` the Copilot and ✨ actions return a **clearly flagged mock narrative** over **real** evidence; the emergency "disable AI" steps in §13 also apply here (no external model calls).
- **Endpoints**: flat under `/api/qynsight/intelligence/*`, Sanctum + `throttle:ai-workspace` (shares the same limiter/budget as Quenyx AI).
- **Auditing**: alert investigations and other AI actions write to `audit_logs` (`action LIKE 'ai%'`); recent investigations also surface on the Operations Intelligence dashboard.
- **Troubleshooting**: a `403 Locked` means the `qynsight` entitlement or `can_use_ai` capability is missing; an empty/limited narrative with "insufficient evidence" means monitoring has not yet collected enough data (check scheduler/agents), **not** an AI failure.

QynAsset Asset Intelligence + AI Adapter Platform (Sprint 22)

- **Endpoints**: flat under `/api/qynasset/intelligence/*`, Sanctum + `throttle:ai-workspace` (shares the Quenyx AI limiter/budget); adapter discovery under `/api/ai/adapters` + `/api/ai/actions`.
- **Entitlement/RBAC**: a `403` means the `qynasset` entitlement, `accessAi` RBAC, or `can_use_ai` capability is missing. UUID-only — a `404 Asset not found` means the asset UUID does not resolve to a host in this workspace.
- **Empty/honest output**: an asset overview with `total: 0` means no hosts/agents are discovered yet (enroll agents / define hosts) — not an AI failure. License/lifecycle-date sections returning `available: false` is **expected** (no inventory/license integration configured), **not** an error; Quenyx AI never fabricates those facts.
- **Auditing**: asset AI actions write to `audit_logs` (`action LIKE 'asset_intelligence_%'` and `ai_adapter_discovery_%`); recent investigations also surface on the Asset Intelligence dashboard.
- **No duplicated AI**: QynAsset reuses the shared `ModuleAiNarrator`/provider runtime; there is no separate provider, prompt, reasoning, or RAG engine to operate.